

L22 — Stack: Basic Terminology, Sequential and Linked Representations

Unit: 2 | Chapter: Stacks & Recursion | BT Level: BT3 | CO: CO2

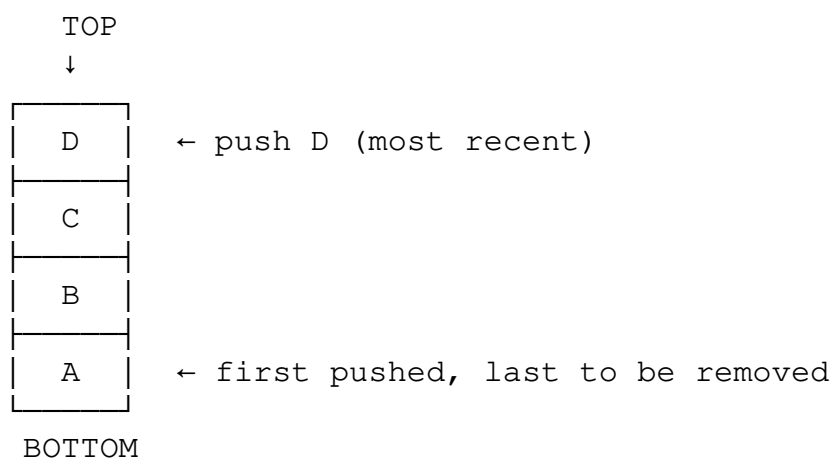
Learning Objectives

- Define a stack and its LIFO principle
 - Implement a stack using arrays (sequential representation)
 - Implement a stack using a linked list
 - Compare both implementations and describe their trade-offs
-

1. What is a Stack?

A **stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle — the element inserted last is the first to be removed.

Real-world analogy: A stack of plates — you add and remove from the top only.



2. Stack Terminology

Term	Definition
TOP	Index/pointer to the topmost element
PUSH	Insert element onto the top of the stack
POP	Remove element from the top of the stack
PEEK / TOP	View top element without removing
isEmpty	Check if stack has no elements
isFull	Check if stack is at maximum capacity (array-based only)
Overflow	Attempting PUSH on a full stack
Underflow	Attempting POP on an empty stack

3. Stack ADT

ADT Stack:

Operations:

<code>push(item)</code>	<code>→ void</code>	– Add item to top
<code>pop()</code>	<code>→ item</code>	– Remove and return top item
<code>peek()</code>	<code>→ item</code>	– Return top item (no removal)
<code>isEmpty()</code>	<code>→ bool</code>	– True if stack is empty
<code>size()</code>	<code>→ int</code>	– Number of elements

4. Array-Based (Sequential) Stack

4.1 Implementation

```
class ArrayStack:
    def __init__(self, capacity=100):
        self.stack = [None] * capacity
        self.capacity = capacity
        self.top = -1      # -1 means empty

    def push(self, item):
        if self.top == self.capacity - 1:
            raise OverflowError("Stack Overflow")
        self.top += 1
        self.stack[self.top] = item

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack Underflow")
        item = self.stack[self.top]
        self.stack[self.top] = None  # Optional: clear slot
        self.top -= 1
        return item

    def peek(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self.stack[self.top]

    def is_empty(self):
        return self.top == -1

    def is_full(self):
        return self.top == self.capacity - 1

    def size(self):
        return self.top + 1

    def display(self):
```

```

print("[", end="")
for i in range(self.top + 1):
    print(self.stack[i], end=", " if i < self.top else "")
print("] ← TOP")

```

4.2 Trace

```

s = ArrayStack()
s.push(10)      # stack: [10]           top=0
s.push(20)      # stack: [10, 20]       top=1
s.push(30)      # stack: [10, 20, 30]   top=2
s.peek()        # returns 30, top=2 (unchanged)
s.pop()         # returns 30, stack: [10, 20], top=1
s.pop()         # returns 20, stack: [10], top=0

```

4.3 Complexity

Operation Time Space

push	O(1)	O(1)
pop	O(1)	O(1)
peek	O(1)	O(1)
isEmpty	O(1)	O(1)

Total space: $O(n)$ for n elements.

5. Linked List Stack

Uses a linked list — no fixed capacity.

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedStack:
    def __init__(self):
        self.top_node = None    # top of stack = head of linked list
        self._size = 0

    def push(self, item):
        new_node = Node(item)
        new_node.next = self.top_node    # New node points to old top
        self.top_node = new_node        # New node becomes top
        self._size += 1

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack Underflow")
        item = self.top_node.data
        self.top_node = self.top_node.next    # Advance top

```

```

        self._size -= 1
        return item

def peek(self):
    if self.is_empty():
        raise IndexError("Stack is empty")
    return self.top_node.data

def is_empty(self):
    return self.top_node is None

def size(self):
    return self._size

def display(self):
    curr = self.top_node
    print("TOP ", end="")
    while curr:
        print(f"[{curr.data}]", end=" → ")
        curr = curr.next
    print("None")

```

5.1 Trace

```

push(10): TOP [10] → None
push(20): TOP [20] → [10] → None
push(30): TOP [30] → [20] → [10] → None
pop():    returns 30, TOP [20] → [10] → None

```

6. Array Stack vs Linked Stack

Property	Array Stack	Linked Stack
Capacity	Fixed (pre-allocated)	Dynamic (unlimited)
Memory	Fixed, may waste	Allocates as needed
Memory overhead	None	Pointer per node (+ 8 bytes)
Cache performance	Excellent	Poor
Overflow possible	Yes	No (memory permitting)
Implementation	Simpler	Slightly complex
push / pop time	O(1)	O(1)

When to use:

- **Array stack:** Known max size, performance-critical, simple implementation
 - **Linked stack:** Unknown/variable size, no memory waste
-

7. Python's Built-in Stack

Python's `list` can be used directly as a stack:

```
stack = []
stack.append(10)    # push
stack.append(20)
stack.append(30)
print(stack[-1])   # peek → 30
stack.pop()        # pop → 30
```

- `list.append()` → $O(1)$ amortized
- `list.pop()` → $O(1)$

Python also has `collections.deque` for a more efficient stack (no resizing overhead).

Summary

- A **stack** follows LIFO: last pushed, first popped.
 - Core operations: push, pop, peek — all $O(1)$.
 - **Array implementation:** $O(1)$ operations, fixed capacity, excellent cache performance.
 - **Linked list implementation:** $O(1)$ operations, dynamic capacity, pointer overhead.
 - Python's `list` serves as a built-in stack.
-

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.1 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.2 (Springer, 2020)